Android Testing — JUnit & Robolectric (Revision Note)

The Split

Tool	What it is	Runs on
JUnit	Test framework — runs tests, provides assertions	JVM
Robolectric	Simulates the Android framework on the JVM	JVM

JUnit is the runner. Robolectric fakes Android. Robolectric plugs into JUnit via `@RunWith` — it is not an alternative to JUnit.

Source Set Layout

```
src/test/          → JVM tests. JUnit (+ Robolectric if Android deps). Fast.
src/androidTest/   → Instrumented tests. Runs on a real device/emulator.
Slow.
```

---

1. JUnit — the foundation

kotlin

```kotlin
class CalculatorTest {

    private lateinit var calculator: Calculator

    @Before
    fun setup() {
        calculator = Calculator()
    }

    @Test
    fun `add returns sum of two numbers`() {
        val result = calculator.add(2, 3)
        assertEquals(5, result)
    }

    @Test(expected = IllegalArgumentException::class)
    fun `divide by zero throws`() {
        calculator.divide(10, 0)
    }

    @After
    fun tearDown() { }
}
```

Annotations

kotlin

```kotlin
@Test                          // marks a test method
@Before                        // runs before EACH test
@After                         // runs after EACH test
@BeforeClass                   // runs ONCE before all tests (static / companio
@AfterClass                    // runs ONCE after all tests (static / companion
@Ignore("reason")              // skips this test

@Test(expected = SomeException::class)   // expects an exception
```

Lifecycle for 3 tests

```
@BeforeClass (once)
  @Before → @Test 1 → @After
  @Before → @Test 2 → @After
  @Before → @Test 3 → @After
@AfterClass (once)
```

Assertions

`assertEquals`, `assertNotEquals`, `assertTrue`, `assertFalse`, `assertNull`, `assertNotNull`, `assertThrows`.

JUnit 4 vs 5

Android still defaults to **JUnit 4**. JUnit 5 renames the lifecycle annotations (`@Before` → `@BeforeEach`, `@BeforeClass` → `@BeforeAll`, `@RunWith` → `@ExtendWith`) and needs an extra Gradle plugin. Know the difference exists.

Backtick test names (`fun `login success updates state`()`) are idiomatic Kotlin and read well in reports.

---

2. Robolectric — Android framework on the JVM

The problem: unit tests run on the JVM, but the `android.jar` available there is a stub — every framework method throws `RuntimeException: Stub! ... not mocked`. So anything calling `Context`, `SharedPreferences`, `Bundle`, `TextUtils`, or resources fails immediately.

Two options: run on a device (slow, needs emulator), or use Robolectric (fast, JVM).

kotlin

```kotlin
@RunWith(RobolectricTestRunner::class)
class PreferencesTest {

    @Test
    fun `saves and reads user name`() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        val prefs = UserPreferences(context)

        prefs.saveName("Hemanth")

        assertEquals("Hemanth", prefs.getName())
    }
}
```

Robolectric supplies working **shadow** implementations of Android classes — real behaviour, no device.

Annotations

kotlin

```
@RunWith(RobolectricTestRunner::class)              // the key one

@Config(sdk = [Build.VERSION_CODES.TIRAMISU])       // pin an SDK level
@Config(application = MyCustomApplication::class)   // custom Application class

@Implements(SomeAndroidClass::class)                // author your own shadow (
```

Most of the time `@RunWith(RobolectricTestRunner::class)` plus `@Test` is all you need.

---

3. When to use which

```
Does the code touch the Android framework?
|
├─ No  (business logic, use cases, utils, pure Kotlin)
|     └─ JUnit alone, src/test — milliseconds per test
|
└─ Yes (Context, SharedPreferences, Bundle, resources, TextUtils)
      └─ JUnit + Robolectric, src/test — seconds per test
```

Push as much as possible into the first bucket.

Rough cost profile:

```
JUnit only (no Android)      → 1000 tests, ~5 seconds
JUnit + Robolectric          →  100 tests, ~50 seconds
Instrumented (on device)     →   10 tests, ~5 minutes
```

Interview Q&A

Q: Did you work on unit testing? "Yes — JUnit and Robolectric. Pure Kotlin code with no Android dependencies runs on the JVM under JUnit. Where there's an Android framework dependency, I use Robolectric for that file."

Q: Why can't you run plain JUnit for Android framework code? "JUnit runs on the JVM, but the `android.jar` provided for the JVM is a stub — every framework method throws `not mocked`. So `Context.getSharedPreferences()` or `TextUtils.isEmpty()` crashes. Robolectric provides shadow implementations that actually work on the JVM, so I get a functioning fake Android environment without a device."

Q: Then why not use Robolectric for everything, instead of two environments? "Speed and scope. Robolectric simulates the whole framework, so it's an order of magnitude slower. A pure JUnit test on business logic runs in milliseconds; the same thing under Robolectric takes seconds. Across thousands of tests that compounds. So JUnit for anything that doesn't touch Android, Robolectric only where I have to."

Q: Unit test vs instrumented test? "Unit tests live in `src/test`, run on the JVM, and are fast. Instrumented tests live in `src/androidTest`, run on a device or emulator, and are slow but exercise the real framework."

Q: How do you test coroutines? "`runTest` from `kotlinx-coroutines-test`, with a `TestDispatcher` injected in place of `Dispatchers.IO`. For `viewModelScope`, set `Dispatchers.setMain(testDispatcher)` in `@Before` and `Dispatchers.resetMain()` in `@After`."

Q: How do you test Flows? "Turbine — `flow.test { assertEquals(expected, awaitItem()) }`. It handles collection and cancellation so you're not fighting the hot/cold distinction manually."

Q: What's your testing strategy? "Most tests as fast JVM unit tests with JUnit, Robolectric only for Android-dependent code, and a small number of instrumented tests for critical user flows."

Quick Reference

JUnit: `@Test`, `@Before`, `@After`, `@BeforeClass`, `@AfterClass`, `@Ignore` **Robolectric:** `@RunWith(RobolectricTestRunner::class)`, `@Config` **Coroutines:** `runTest`, `TestDispatcher`, `Dispatchers.setMain/resetMain` **Flows:** Turbine — `awaitItem()`, `awaitComplete()`, `awaitError()`

One line: **JUnit provides the lifecycle. Robolectric fakes Android.**